

Project report on: Instance Segmentation using Mask-RCNN

Instructor: Dr. Muhammad Animul Islam

Course: Deep Learning (DSCI-6011-02)

Team members:

1. Suraj Thapa
2. Abhigya Malla



University of New Haven

300 Boston Post Road
West Haven, CT 06516

1. Introduction

Instance segmentation is a deep learning–based computer vision task that aims to identify, classify, and precisely segment each individual object instance within an image. Unlike semantic segmentation, which assigns a single class label to all pixels of the same category, or object detection, which predicts only bounding boxes around objects, instance segmentation provides pixel-accurate masks for every distinct object, even when multiple objects belong to the same class. This combination of fine-grained localization and pixel-level understanding makes instance segmentation essential in domains where accuracy is critical, such as autonomous driving, medical imaging, aerial analysis, and robotics.

However, instance segmentation is inherently challenging because it requires solving two problems simultaneously:

1. Detecting all objects present in the image, and
2. Segmenting each detected object at the pixel level.

This dual requirement demands a model architecture that can balance detection precision with dense prediction quality.

A widely adopted framework for this task is Mask R-CNN, which we employ in this project. Mask R-CNN extends Faster R-CNN by adding a parallel mask-prediction branch that generates a binary segmentation mask for each Region of Interest (RoI). This mask branch is implemented as a lightweight fully convolutional network (FCN), making the overall architecture straightforward to train and efficient to experiment with. Despite its simplicity, Mask R-CNN consistently achieves strong performance across a wide variety of instance segmentation benchmarks.

The motivation for choosing this topic stems from two factors. First, instance segmentation unifies the concepts of object detection and semantic segmentation, allowing us to explore and understand both tasks within a single framework. Second, instance segmentation has significant practical relevance, being used in fields where object-level understanding and high spatial precision are crucial. Mask R-CNN is particularly suited for this project due to its accuracy, modular design, and proven effectiveness in real-world applications, despite its relatively high computational cost.

In this project, we build an instance segmentation system for three everyday object categories—pen, plug, and shoe—collected from real images. We adapt Mask R-CNN to our custom dataset, apply extensive data augmentation, perform hyperparameter tuning, fine-tune the model, and evaluate its performance using metrics such as mAP and IoU. We also explore a recent enhancement, Test-Time Augmentation (TTA), to further improve prediction quality.

2. Method

In this project, we use Mask R-CNN as the primary architecture for instance segmentation. Mask R-CNN extends Faster R-CNN by adding a fully convolutional mask-prediction branch that produces a pixel-wise segmentation mask for each detected Region of Interest (RoI). The system consists of four main components: the backbone network, the Region Proposal Network (RPN), the RoI Align layer, and the mask head.

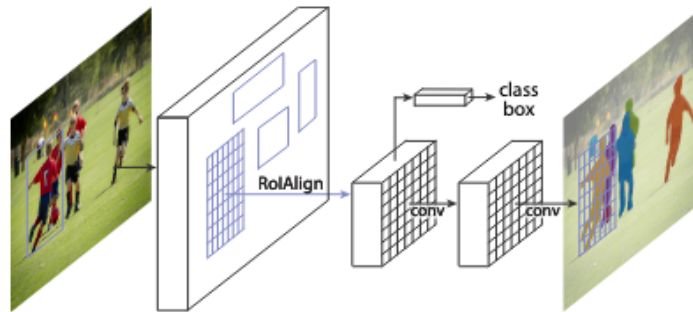


Figure 1. The Mask R-CNN framework for instance segmentation.

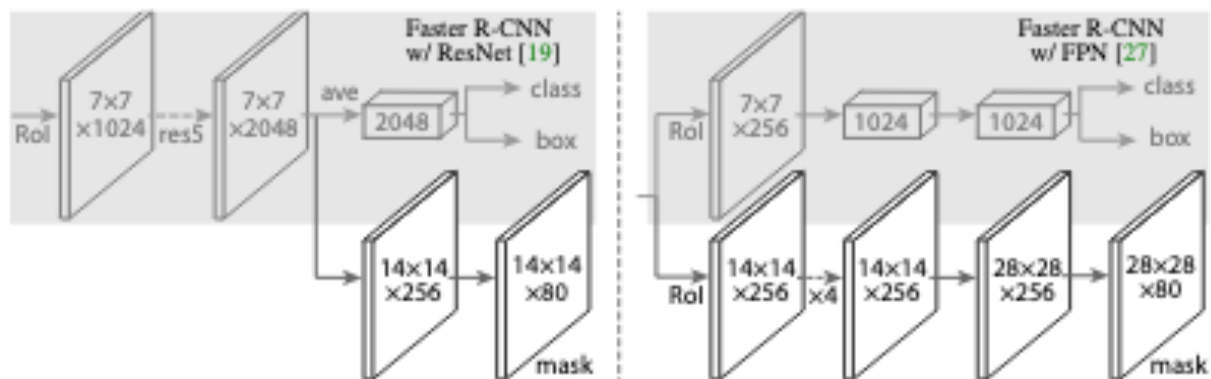


Figure 2. Head Architecture: Kaiming and his team from Facebook AI Research (FAIR) extend two Faster R-CNN heads. Left/Right panels show the heads for the ResNet C4 and FPN backbones, to which a mask branch is added. Numbers denote spatial resolution and channels. Arrows denote either conv, deconv, or fc layers as can be inferred from context (conv preserves spatial dimension while deconv increases it). All convs are 3×3 , except the output conv which is 1×1 , deconvs are 2×2 with stride 2, and they used ReLU in hidden layers. Left: ‘res5’ denotes ResNet’s fifth stage, which for simplicity we altered so that the first conv operates on a 7×7 RoI with stride. Right: ‘ $\times 4$ ’ denotes a stack of four consecutive convs.

2.1 Mask R-CNN Architecture Overview

Backbone Network

Mask R-CNN uses a convolutional backbone to extract multi-scale features from the input image. In the original implementation, two backbone variants are used:

- ResNet-C4 (features extracted from the 4th residual block)
- ResNet-FPN (ResNet with a Feature Pyramid Network)

In our project, we adopt the ResNet-50 FPN backbone (via maskrcnn_resnet50_fpn_v2) because FPN provides richer multi-scale features and significantly improves performance on small or thin objects (such as pens and plugs). FPN constructs a top-down feature hierarchy, upsampling deeper feature maps and merging them with earlier ones to produce pyramid levels P2–P6.

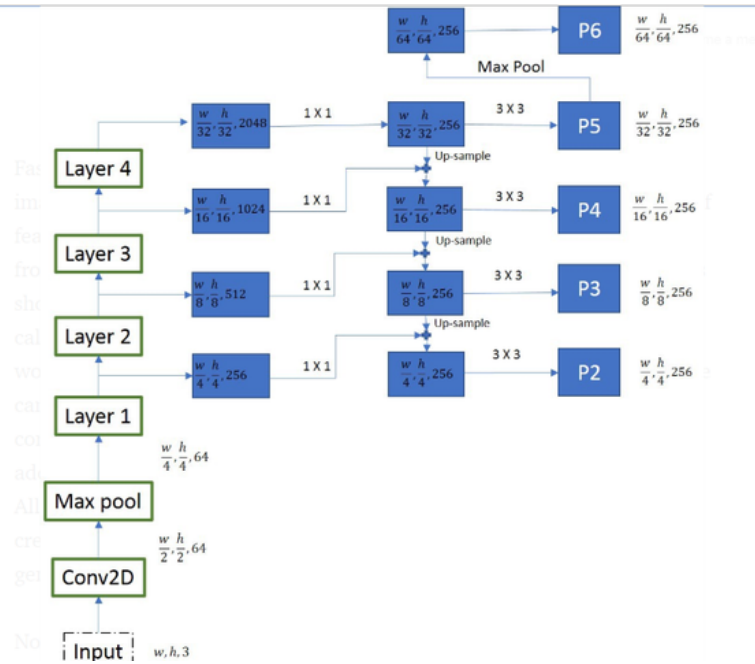


Figure 3. Backbone network

2.2 Region Proposal Network (RPN)

The RPN operates on the FPN feature maps and predicts:

1. Objectness scores (is there an object?)
2. Bounding box offsets (anchor refinement)

Anchors at multiple FPN scales allow the network to detect objects of various sizes without manually designing anchor ratios for each scale.

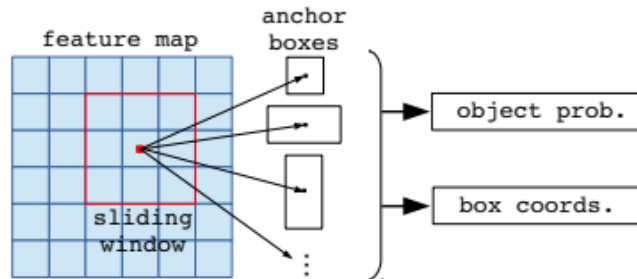


Figure 4. Anchor Generation Mask R-CNN

2.3 RoI Align

Traditional RoIPool used quantization, which created misalignment between RoIs and feature maps.

Mask R-CNN introduces RoIAlign, which:

- avoids quantization,
- uses bilinear interpolation, and
- produces a fixed-size feature map for every RoI.

This alignment is crucial for accurate pixel-level mask prediction.

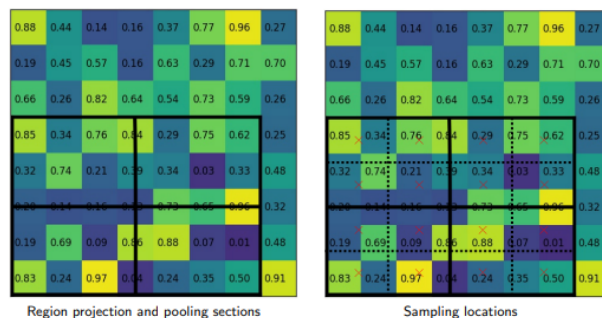


Figure 5. ROI Align

Given the feature map of the previous Convolution layer of size $h \times w$, divide this feature map into $M \times N$ grids of equal size (NOT just take integer value).

The mask R-CNN inference speed is around 2 fps, which is good considering the addition of a segmentation branch in the architecture.

2.4 Mask Head

The mask head is a lightweight Fully Convolutional Network (FCN) applied to each RoI.

It predicts an $m \times m$ spatial mask (typically 28×28 for FPN) for each class as shown in Figure 2.

Importantly:

- A per-pixel sigmoid is used.
- Only the mask of the ground-truth class contributes to the mask loss.
- Mask prediction is decoupled from classification (unlike semantic segmentation softmax).

This design improves mask quality and avoids class-wise competition.

2.5 Loss Function

Mask R-CNN employs a multi-task loss for each RoI:

$$L = L_{\text{cls}} + L_{\text{box}} + L_{\text{mask}}$$

- Classification loss (L_{cls}): softmax cross-entropy
- Box regression loss (L_{box}): Smooth L1 loss
- Mask loss (L_{mask}): binary cross-entropy applied pixel-wise to the target mask

Only one mask channel (the ground-truth class) contributes to the loss, enabling the network to learn class-specific masks independently.

2.6 Network Modifications for Our Task

Mask R-CNN is originally designed for COCO with 80 classes.

To adapt it to our dataset, we modify only the final heads:

- Box predictor: replaced with a new predictor having 4 output classes (background, pen, plug, shoe)
- Mask predictor: similarly updated to produce 4-class masks

No other structural changes were made, preserving the standard Mask R-CNN pipeline.

3. Dataset

3.1 Data Collection

The dataset used in this project consists of 200+ real-world images, collected manually using a smartphone camera. Each image contains at least one instance of the target object categories:

- Pen
- Plug
- Shoe

The images were captured under natural lighting conditions, different backgrounds, and varying object distances to ensure diversity. This real-world variability is important for training a robust instance segmentation model.

Each image was manually annotated using polygon masks to generate pixel-accurate object boundaries. The annotations follow the standard Roboflow JSON format, and are converted into Mask R-CNN-compatible masks, bounding boxes, and labels during preprocessing.

3.2 Dataset Partitioning

After annotation, the dataset was divided into:

- 80% Training set
- 10% Validation set
- 10% Test set

Partitioning was performed using randomized sampling to ensure that all object classes were represented in each split.

```
# Get image keys
```

```
img_keys = list(img_dict.keys())
```

```
random.shuffle(img_keys)
```

```
n = len(img_keys)
```

```
train_split = int(0.8 * n)
```

```
val_split    = int(0.9 * n)
```

```
train_keys = img_keys[:train_split]
val_keys    = img_keys[train_split:val_split]
test_keys   = img_keys[val_split:]
```

This ensures unbiased evaluation of model performance before and after fine-tuning.

3.3 Data Augmentation

Because our dataset is relatively small, we apply extensive data augmentation to increase diversity and improve generalization. Augmentation is applied only to training images, never to validation or test samples.

The following augmentations were used (as implemented in the code):

- Random IoU crop: crops random regions that still contain objects
- Color jitter: changes brightness, contrast, saturation, and hue
- Random grayscale
- Random equalization
- Random posterization
- Horizontal flipping
- Multi-scale resizing (random scale between 640–800 px)
- Pad-to-square for consistent input size
- Final resize to 256×256

Augmentation code snippet:

```
data_aug_tfms = T.Compose([
    iou_crop,
    T.ColorJitter(
        brightness=(0.875, 1.125),
        contrast=(0.5, 1.5),
        saturation=(0.5, 1.5),
        hue=(-0.05, 0.05),
```



```

    ),
    T.RandomGrayscale(p=0.1),
    T.RandomEqualize(p=0.2),
    T.RandomPosterize(bits=3, p=0.5),
    T.RandomHorizontalFlip(p=0.5),
])

```

These augmentations significantly help the model detect objects under various lighting, orientations, and background conditions.

3.4 Preprocessing and Normalization

Before feeding images into the model, several preprocessing steps are applied:

1. Resize to a random scale (640–800 px)

```

scale = random.randint(640, 800)
resize_max = ResizeMax(max_sz=scale)

```

2. Pad to a square

Ensures consistent input dimensions:

```

pad_square = PadSquare(shift=True, fill=0)

```

3. Final resize to training size (256×256)

```

T.Resize([train_sz, train_sz], antialias=True)

```

4. Convert to tensor & normalize to [0,1]

```
final_tfms = T.Compose([
    T.ToImage(),
    T.ToDtype(torch.float32, scale=True),    # normalizes to [0,1]
    T.SanitizeBoundingBoxes(),
])
```

Full training transform:

```
train_tfms = T.Compose([
    data_aug_tfms,
    resize_max,
    pad_square,
    T.Resize([256, 256], antialias=True),
    final_tfms
])
```

Validation & test transforms:

No augmentation is applied.

```
valid_tfms = T.Compose([
    resize_max,
    pad_square,
    T.Resize([256,256], antialias=True),
    final_tfms
])
```

3.5 Annotation Structure

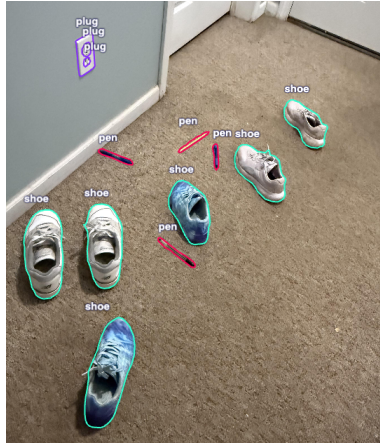


Figure 6. Sample annotated image

Annotation data of Figure 6. is:

```
{
  "boxes": [
    {
      "id": "1",
      "type": "polygon",
      "label": "plug",
      "x": "700.5000",
      "y": "950.5000",
      "width": "137.0000",
      "height": "313.0000",
      "points": [ [638,852], [632,858], [637,881], [651,1047],
[658,1096], [667,1107], [679,1104], [729,1071], [759,1048],
[769,1033], [753,799], [750,794], [737,795]]
    }
  ]
}
```

```

    },
    {
        "id": "4",
        "type": "polygon",
        "label": "plug",
        "x": "709.0000",
        "y": "916.5000",
        "width": "62.0000",
        "height": "105.0000",
        "points": [[678,912], [678,943], [689,960], [714,969],
[737,946], [740,921], [735,907], [
736,881], [726,864], [709,870], [696,880]]
    },
    {
        "id": "5",
        "type": "polygon",
        "label": "plug",
        "x": "713.0000",
        "y": "1006.0000",
        "width": "48.0000",
        "height": "66.0000",
        "points": [[694,988], [689,1013], [690,1029], [702,1039],
[730,1017], [737,999], [726,973], [702,979]]
    },

```

```

{
  "id": "6",
  "type": "polygon",
  "label": "pen",
  "x": "917.5000",
  "y": "1680.5000",
  "width": "245.0000",
  "height": "115.0000",
  "points": [[797,1637], [795,1645], [820,1664], [886,1685],
[974,1724], [1007,1735], [1030,1738], [1035,1737], [1040,1729],
[1036,1716], [1021,1701], [957,1679], [821,1623], [802,1629]]
},
{
  "id": "7",
  "type": "polygon",
  "label": "pen",
  "x": "1474.5000",
  "y": "1576.5000",
  "width": "217.0000",
  "height": "147.0000",
  "points": [[1374,1629], [1366,1645], [1375,1650],
[1387,1647], [1528,1553], [1539,1542], [1552,1537], [1561,1527],
[1576,1519], [1583,1504], [1564,1503], [1542,1513], [1531,1524],
[1494,1540], [1483,1551], [1447,1571], [1422,1596], [1399,1606]]
},
{

```

```

        "id": "8",

        "type": "polygon",

        "label": "pen",

        "x": "1631.0000",

        "y": "1681.0000",

        "width": "42.0000",

        "height": "184.0000",

        "points": [[1618,1597], [1610,1613], [1610,1623],
[1621,1677], [1622,1742], [1626,1760], [1636,1773], [1650,1756],
[1652,1737], [1646,1614], [1643,1598], [1636,1589], [1624,1591]]

    },

    {

        "id": "9",

        "type": "polygon",

        "label": "pen",

        "x": "1352.5000",

        "y": "2363.0000",

        "width": "263.0000",

        "height": "212.0000",

        "points": [[1221,2260], [1222,2280], [1271,2327],
[1315,2351], [1336,2372], [1458,2466], [1472,2469], [1484,2465],
[1484,2447], [1471,2428], [1458,2417], [1410,2383], [1365,2357],
[1253,2268], [1235,2257]]

    },

    {

        "id": "A",

```

```

    "type": "polygon",

    "label": "shoe",

    "x": "875.5000",

    "y": "3218.5000",

    "width": "357.0000",

    "height": "785.0000",

    "points": [[703,3393], [697,3458], [697,3508], [702,3554],
[713,3583], [728,3598], [750,3607], [809,3611], [833,3603],
[867,3577], [883,3559], [900,3535], [915,3502], [927,3458],
[943,3420], [1000,3227], [1032,3156], [1050,3095], [1054,3019],
[1044,2955], [1018,2890], [993,2858], [963,2831], [936,2826],
[918,2834], [870,2867], [843,2901], [813,2968], [793,3030],
[790,3051], [725,3205], [722,3219], [724,3259], [715,3333]]
  },
  {

    "id": "B",

    "type": "polygon",

    "label": "shoe",

    "x": "401.5000",

    "y": "2375.5000",

    "width": "279.0000",

    "height": "635.0000",

    "points": [[266,2249], [262,2293], [268,2330], [268,2389],
[276,2469], [281,2488], [301,2528], [321,2616], [336,2637],
[348,2664], [363,2677], [386,2689], [404,2693], [454,2687],
[497,2658], [513,2636], [525,2604], [533,2559], [533,2505],
[541,2449], [541,2389], [530,2295], [535,2208], [526,2152],
[513,2117], [493,2086], [465,2065], [443,2059], [416,2058],
[373,2068], [347,2086], [319,2116], [286,2181]]
  }
}

```

```

    },
    {
        "id": "C",
        "type": "polygon",
        "label": "shoe",
        "x": "826.5000",
        "y": "2318.0000",
        "width": "271.0000",
        "height": "602.0000",
        "points": [[728,2071], [710,2148], [702,2164], [691,2165],
[716,2212], [721,2271], [714,2349], [718,2422], [735,2462],
[758,2546], [782,2591], [797,2606], [812,2614], [838,2619],
[870,2615], [902,2600], [915,2589], [936,2559], [944,2532],
[953,2437], [962,2391], [962,2179], [953,2150], [918,2082],
[895,2051], [867,2027], [822,2017], [773,2022], [749,2038]]
    },
    {
        "id": "D",
        "type": "polygon",
        "label": "shoe",
        "x": "1444.0000",
        "y": "2077.5000",
        "width": "296.0000",
        "height": "457.0000",
        "points": [[1296,1975], [1296,2023], [1304,2050],
[1362,2133], [1411,2237], [1446,2291], [1458,2299], [1481,2306],
[1537,2300], [1553,2294], [1565,2282], [1574,2259], [1582,2180],

```



```

[1588,2166], [1592,2138], [1575,2022], [1553,1961], [1531,1938],
[1435,1864], [1397,1849], [1364,1855], [1352,1864], [1324,1904]]

    },

    {

        "id": "E",

        "type": "polygon",

        "label": "shoe",

        "x": "1949.0000",

        "y": "1798.5000",

        "width": "364.0000",

        "height": "407.0000",

        "points":
[[1767,1685],[1768,1720],[1775,1755],[1798,1793],[1841,1834],[1888,186
6],[1909,1890],[1931,1928],[1985,1982],[2036,1999],[2073,2002],[2097,1
992],[2118,1976],[2127,1950],[2122,1917],[2123,1876],[2131,1852],[2129
,1835],[2110,1812],[2099,1790],[2085,1708],[2054,1676],[2025,1668],[20
20,1663],[2013,1635],[2002,1635],[1986,1651],[1932,1643],[1895,1614],[
1870,1600],[1851,1595],[1823,1595],[1813,1598],[1794,1614],[1788,1632]
,[1778,1646]]

    },

    {

        "id": "F",

        "type": "polygon",

        "label": "shoe",

        "x": "2275.0000",

        "y": "1446.5000",

        "width": "310.0000",

```

```

        "height": "343.0000",

        "points":
[[2120,1349],[2121,1391],[2131,1413],[2153,1441],[2180,1462],[2206,147
5],[2221,1489],[2237,1517],[2251,1566],[2275,1596],[2309,1612],[2356,1
618],[2382,1606],[2391,1606],[2403,1596],[2408,1584],[2406,1553],[2412
,1517],[2430,1472],[2422,1455],[2404,1451],[2389,1441],[2368,1395],[23
54,1377],[2267,1325],[2228,1292],[2183,1275],[2167,1275],[2142,1293],[
2134,1316],[2123,1330]]

    }

],

    "height": 4032,

    "key": "00195.jpg",

    "width": 3024

}

```

4. Experiments and results

This section presents the experimental setup, hyperparameter search procedure, training results, performance evaluation, and the impact of applying a recent enhancement technique—Test-Time Augmentation (TTA)—to the Mask R-CNN model.

4.1 Experimental Setup

All experiments were conducted using the PyTorch implementation of Mask R-CNN with a ResNet-50 FPN backbone. Training and inference were executed on Google Colab with GPU acceleration. The model was initialized with pretrained weights (ImageNet backbone and COCO-trained heads) and then adapted to our custom dataset containing four classes: background, pen, plug, and shoe. Extensive data augmentation was applied only to the training set to improve robustness and reduce overfitting. Validation and test images were only resized, padded, and normalized.

4.2 Hyperparameter Tuning

To identify an effective learning configuration, we performed a small grid search over learning rate and weight decay. Due to limited compute resources, the search was intentionally small but still systematic.

Hyperparameter Grid

Learning rates tested: [1e-3, 5e-4]

Weight decays tested: [1e-4]

Epochs per trial:1

Best Hyperparameters Found

Learning rate: 0.0005

Weight decay: 0.0001

The learning rate 5e-4 consistently produced lower validation loss and slightly higher mAP/IoU during tuning. These values were used for full fine-tuning.

```
lrs = [1e-3, 5e-4]
```

```
weight_decays = [1e-4]
```

```
epoch_options = [1]
```

```
best_cfg = None
```

```
best_score = -1.0 # e.g., best mAP@0.50
```

```
for lr in lrs:
```

```
    for wd in weight_decays:
```

```
        for num_epochs in epoch_options:
```

```
            # fresh model each time
```

```
            model = create_model(num_classes=len(class_names),
```

```
                                device=device, dtype=torch.float32)
```

```

optimizer = torch.optim.AdamW(model.parameters(),
                                lr=lr, weight_decay=wd)

lr_sched = torch.optim.lr_scheduler.OneCycleLR(
    optimizer, max_lr=lr,
    total_steps=num_epochs * len(train_dataloader)
)

train_loss = run_epoch(model, train_dataloader,
                        optimizer, lr_sched,
                        device, scaler=None,
                        epoch_id=0, is_training=True)

metrics = evaluate_instance_segmentation(
    model, valid_eval_loader, device
) # returns dict with 'map_50' etc.

if metrics["map_50"] > best_score:
    best_score = metrics["map_50"]
    best_cfg = {"lr": lr, "weight_decay": wd,
               "epochs": num_epochs}

```

4.3 Training vs. Pre-Training Performance

Before fine-tuning, we evaluated the pretrained Mask R-CNN on our dataset to establish a baseline. Pre-training results revealed:

- Very high loss (train ≈ 3.82 , val ≈ 3.78)
- $\text{mAP}@0.50 = 0$
- Mean IoU ≈ 0
- Model failed to detect any pen or plug instances

- Shoe class was rarely detected but masks were inaccurate

This was expected because the COCO-pretrained head was not specialized for our custom object categories.

```
# model with pretrained COCO weights, heads adapted to 4 classes
```

```
model = create_model(num_classes=len(class_names),
                     device=device, dtype=torch.float32)
```

```
baseline_train_metrics = evaluate_instance_segmentation(
    model, train_eval_loader, device
)
```

```
baseline_valid_metrics = evaluate_instance_segmentation(
    model, valid_eval_loader, device
)
```

After fine-tuning on our dataset for 10 epochs using the tuned hyperparameters, the model achieved:

- Train loss ≈ 0.60
- Validation loss ≈ 0.77
- mAP@0.50 (train) ≈ 0.0932
- mAP@0.50 (val) ≈ 0.1186
- Mean IoU (train) ≈ 0.79
- Mean IoU (val) ≈ 0.797

This represents a significant improvement, both qualitatively and quantitatively.

```
best_lr = 5e-4
```

```
best_wd = 1e-4
```

```
epochs = 10
```

```
model = create_model(num_classes=len(class_names),
                     device=device, dtype=torch.float32)
```

```
optimizer = torch.optim.AdamW(model.parameters(),
                                lr=best_lr, weight_decay=best_wd)
```

```
lr_scheduler = torch.optim.lr_scheduler.OneCycleLR(
    optimizer, max_lr=best_lr,
    total_steps=epochs * len(train_dataloader)
)
```

```
train_loop(model=model,
            train_dataloader=train_dataloader,
            valid_dataloader=valid_dataloader,
            optimizer=optimizer,
            lr_scheduler=lr_scheduler,
            device=device,
            epochs=epochs,
            checkpoint_path=checkpoint_path,
            use_scaler=True)
```

4.4 Performance Visualization

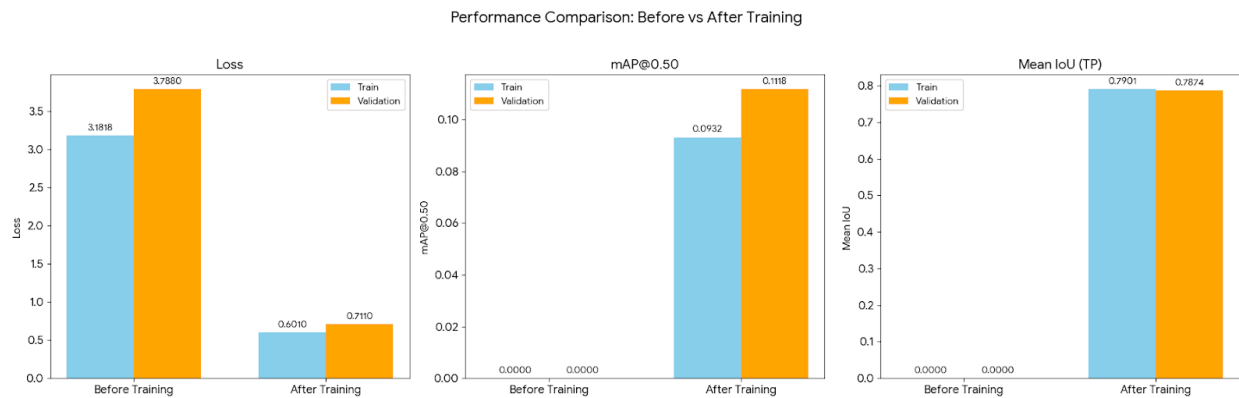


Figure 7. Plot of training and validation performances including loss, mAP, and IoU.

The three plots in Figure 7 summarize the change in performance before and after fine-tuning.

Performance Comparison: Before vs After Training

- Loss: Large drop for both train and validation splits
- mAP@0.50: Improves from 0 to 0.09–0.12
- Mean IoU (TP): Improves from 0 to ≈ 0.80

These results confirm that the model successfully learned pixel-level masks and bounding boxes for at least one object class.

4.5 Class-Wise Performance Discussion

During initial evaluation, the model was trained and tested using a score threshold of 0.5, meaning that the network only retained detections whose confidence score exceeded 50%.

Because pen and plug are smaller, less frequent, and harder to learn, many of their detections fell below this threshold and were therefore discarded, resulting in poor recognition of these classes.

However, after reducing the score threshold to 0.2–0.3, the model began correctly identifying all three classes—pen, plug, and shoe—indicating that the model had learned meaningful features for each class but was previously filtering out lower-confidence detections.

This behavior highlights several underlying factors:

1. Class Imbalance – The dataset contains significantly more shoe samples than pen or plug, causing the model to become biased toward the dominant class.
2. Small Object Size – Pens and plugs occupy fewer pixels and exhibit less texture variation, making them harder for the model to detect confidently.
3. Limited Training Duration – The model was fine-tuned for only 10 epochs due to computational constraints, which may not be sufficient for smaller or underrepresented objects.
4. Restricted Hyperparameter Search – Only a small subset of learning rates and weight decay values was explored, reducing the chance of finding the optimal configuration.
5. Background Similarity – Thin objects like pens often blend with the background, leading to reduced confidence scores.

Overall, lowering the score threshold improved class-wise detection, confirming that the model has the capacity to detect all classes. Nonetheless, there is substantial room for improvement. Increasing data diversity, applying balanced sampling, extending training epochs, and conducting a more comprehensive hyperparameter search would further strengthen performance, particularly for pen and plug detection.

Furthermore, during visualization in the Gradio interface, we applied an **additional post-processing constraint** by keeping only the top two highest-confidence bounding boxes per class using class-wise **Non-Maximum Suppression**. This ensures that the displayed results remain clean and interpretable, avoiding the clutter of multiple overlapping predictions while still preserving the most reliable detections for each object class.

4.6 Recent Technique: Test-Time Augmentation (TTA)

To further improve predictions, we implemented horizontal-flip Test-Time Augmentation, a modern technique often used in segmentation models.

TTA procedure:

1. Predict on the original image
2. Predict on a horizontally flipped version
3. Flip predictions back
4. Merge predictions using Non-Maximum Suppression (NMS)
5. Average masks and scores

Code snippet:

```
def predict_with_tta(model, image_tensor, device, score_thresh=0.5):
    """
    image_tensor: (1, C, H, W) in [0,1]
    """
    model.eval()
    with torch.no_grad():
        # 1) original
        out_orig = model(image_tensor.to(device))[0]

        # 2) flipped
        img_flipped = torch.flip(image_tensor, dims=[3])
        out_flip = model(img_flipped.to(device))[0]

        # flip boxes and masks back
```



```

W = image_tensor.shape[-1]
boxes_f = out_flip["boxes"].clone()
boxes_f[:, [0, 2]] = W - boxes_f[:, [2, 0]] # x1,x2 swap
masks_f = torch.flip(out_flip["masks"], dims=[3])

# 3) concatenate predictions
boxes = torch.cat([out_orig["boxes"], boxes_f], dim=0)
scores = torch.cat([out_orig["scores"], out_flip["scores"]],
dim=0)
labels = torch.cat([out_orig["labels"], out_flip["labels"]],
dim=0)
masks = torch.cat([out_orig["masks"], masks_f], dim=0)

# 4) NMS merge
keep = torchvision.ops.nms(boxes, scores, iou_threshold=0.5)
boxes = boxes[keep]
scores = scores[keep]
labels = labels[keep]
masks = masks[keep]

# filter low confidence
keep2 = scores > score_thresh
return {
    "boxes": boxes[keep2].cpu(),
    "scores": scores[keep2].cpu(),
    "labels": labels[keep2].cpu(),
    "masks": masks[keep2].cpu(),

```

}

Motivation

TTA helps handle:

- noisy inputs
- orientation variance
- small or occluded objects
- inconsistent lighting

Result

In our dataset:

- TTA did not result in performance improvement
- mAP and IoU were unchanged or slightly worse
- Shoe detection remained correct
- Pen and plug still not detected

Reason

TTA enhances predictions only if the base model is already strong.

Since our model struggled to learn pen/plug class features when score threshold is 0.5, TTA could not compensate for insufficient segmentation quality.

4.7 Summary of Experimental Findings

- Fine-tuning significantly improved segmentation metrics
- However, the small dataset limited class-wise learning
- Shoe class learned well ($\text{IoU} \approx 0.8$), also pen/plug were identified under less threshold values like 0.2 or 0.3.
- Hyperparameter tuning (learning rate $5e-4$, weight decay $1e-4$) improved convergence
- TTA did not improve accuracy due to weak base performance for minority classes
- Additional data or class-balanced augmentation is needed for multi-class improvement

5. Useful links

Link to dataset:

<https://drive.google.com/drive/folders/1rgXpOqdZQEemoX1MMP9XY8V30HNnJDkH?usp=sharing>

Reference paper: <https://arxiv.org/abs/1703.06870>

Link to code:

https://colab.research.google.com/drive/1SAtkuTkwQOt6vrb2WGVAmJcD9CfXG1D8?usp=drive_link

Reference code:

<https://github.com/multimodallearning/pytorch-mask-rcnn/tree/master>

<https://christianjmills.com/posts/pytorch-train-mask-rcnn-tutorial/>